Archived Web Pages & the Wayback Machine

Week 7 · Documents module · Textbook Chapter 7

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

September 28, 30 & October 2, 2026

This week at a glance

The web is not permanent. This week we scrape the *past* — and measure how pages change over time.

Monday | Concepts & notebook intro

- Link rot, the Internet Archive, and two APIs into the past

Wednesday | Notebook lab

- Work the Chapter 7 notebook: Availability API, CDX, tracking a Terms-of-Service document

Friday | Textbook revisions

- Propose & peer-review pull requests improving Chapter 7

Companion notebook

`ch-07-archives.ipynb`

Bring a laptop

Wednesday is hands-on — we query the Archive live.

1. Monday · Concepts

The web is not permanent

Pages change, disappear, and get redesigned constantly. A large fraction of URLs break within a few years — **link rot**.

This is a research problem. How do you study something that can disappear tomorrow? How do you measure a *change* when you can see only today's version?

The **Internet Archive** (Brewster Kahle, 1996) answers by archiving the web. Its **Wayback Machine** has captured over 800 billion pages, free to anyone.

It is both a data source and *counter-erosion infrastructure* — a community-governed public resource (the book's "post-API" ownership theme).

The Wayback calendar: every dot is a capture you can retrieve.

Textbook Ch. 7, "The Web Is Not Permanent."

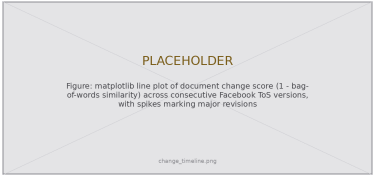
From snapshots to a longitudinal source

One old snapshot is a curiosity. **A comparison of snapshots over time** is what supports research.

The Wayback Machine turns the web from a snapshot medium into a **longitudinal data source** — you can *measure* change, not merely observe it:

- Track a platform's Terms of Service across a decade
- Follow a government page across administrations
- Contrast two rivals' homepages as they evolve

You are not just reading old pages — you are measuring **institutional behavior over time.**



A "change timeline" we build Wednesday: spikes mark major revisions.

Textbook Ch. 7, "Comparing Multiple Snapshots" & "Tracking Policy Changes."

Raw or wrapped? The `id_` rule

A snapshot URL looks like:

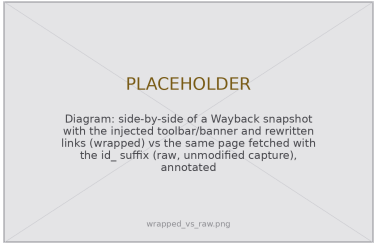
`web.archive.org/web/20000815052826/http://cnn.com/`

By default the Archive **injects** its own toolbar, banner, and JavaScript, and **rewrites** every link, script, and image to point back into the archive. Convenient for browsing — **poison for measurement**: your link, script, and image counts include the scaffolding.

Append `id_` to the timestamp

`.../20000815052826id_/http://cnn.com/`

to get the **original, unmodified capture**. Raw doesn't render prettily — but for measurement, raw is exactly what you want.



Wrapped (toolbar + rewritten links) vs. the `id_` raw capture.

Textbook Ch. 7, "The Availability API." See also Brügger (2019) on archived-web caveats.

In the notebook

This week's companion notebook builds one skill at a time:

- **Availability API —** find the snapshot closest to a date; retrieve it raw with `id_`
- **CDX Server API —** enumerate a URL's *entire* capture history, filtered server-side
- **Policy tracking —** loop over quarterly dates, collect a document's history, serialize to JSON
- **Bag-of-words —** tokenize, drop stopwords, and measure how much consecutive versions changed

Connecting to Ch. 6 & what's next

Archived pages parse with `BeautifulSoup` exactly like live ones (Ch. 6), and the same `time.sleep()` politeness applies — the Archive is a nonprofit. The JavaScript growth we measure is *why* dynamic-page techniques (next week) exist.

2. Wednesday · Notebook Lab

Finding a snapshot — the Availability API

The simplest question: does a snapshot exist near a date? Pass query args as a `params` dict so `requests` URL-encodes them (one value is itself a URL):

```
import requests

api_url = "https://archive.org/wayback/available"
response = requests.get(api_url, params={"url": "https://www.cnn.com",
                                         "timestamp": "20000815"}) # Aug 15, 2000
data = response.json()

if data["archived_snapshots"]:
    snap = data["archived_snapshots"]["closest"] # closest capture to that date
    print(snap["timestamp"], snap["url"])
else:
    print("No snapshot available near this date")
```

Textbook Ch. 7, “The Availability API.”

Retrieve it raw with `id_`, then measure

Rewrite the snapshot URL to the `id_` form, then parse it like any page:

```
from bs4 import BeautifulSoup

ts = snap["timestamp"]
raw_url = snap["url"].replace(ts, ts + "id_") # id_ -> original, unwrapped capture

soup = BeautifulSoup(requests.get(raw_url).text, "html.parser")
stats = {"year": ts[:4], "num_links": len(soup.find_all("a")),
         "num_scripts": len(soup.find_all("script"))}
```

The same measurement across three eras tells the web's story:

```
# year num_links text_length num_images num_scripts
# 2000 150 8500 20 2 # largely static HTML
# 2024 800 45000 120 95 # a JS app that renders
```

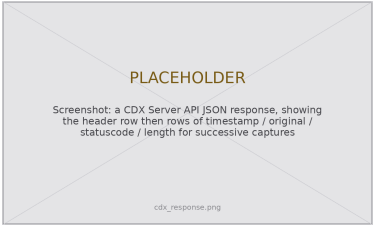
Textbook Ch. 7, "Comparing Multiple Snapshots." Without `id_`, every count is inflated by the toolbar.

The whole history — the CDX Server API

Availability finds *one* snapshot; CDX enumerates *all* of them. Two params filter server-side — less data, less cleanup:

```
cdx = "https://web.archive.org/cdx/search/cdx"
params = {"url": "facebook.com/terms",
          "output": "json",
          "fl": "timestamp,original,statuscode,length",
          "filter": "statuscode:200", # only 200s
          "collapse": "digest"} # drop repeats
data = requests.get(cdx, params=params).json()

df = pd.DataFrame(data[1:], columns=data[0])
df["timestamp"] = pd.to_datetime(
    df["timestamp"], format="%Y%m%d%H%M%S")
```



Row 0 is the header; each later row is one distinct capture.

Textbook Ch. 7, "The CDX Server API." Add `"limit": 100` while developing.

Tracking a document over time

Wrap the Availability + `id_` + parse steps in `get_wayback_content()` (returns `None` on a miss), then loop over quarterly dates:

```
dates = pd.date_range(start="2005-01-01", end="2024-01-01", freq="QS")

results = []
for date in dates:
    snap = get_wayback_content("facebook.com/terms", date)
    if snap is not None: # skip misses instead of crashing the run
        results.append(snap)
    time.sleep(1) # rest between requests -- the Archive is a nonprofit
# Retrieved 68 snapshots from 77 time points
```

Serialize immediately — you may not be able to re-scrape it later:

```
import json
with open("fb_tos_history.json", "w") as f:
    json.dump(results, f, indent=2) # (also cache each raw capture as you go)
```

Textbook Ch. 7, “Retrieving Historical Content,” “Serializing,” & “Caching Raw Captures.”

Measuring change — bag-of-words

Represent each version as a vector of word frequencies (order ignored), then compare vectors:

```
from gensim.utils import simple_preprocess
from gensim import corpora, similarities
from nltk.corpus import stopwords # nltk.download("stopwords") once

stop = set(stopwords.words("english"))
docs = [[t for t in simple_preprocess(r["content"]) if t not in stop] for r in results]

dct = corpora.Dictionary(docs) # word -> id
corpus = [dct.doc2bow(d) for d in docs] # each version -> word-count vector
index = similarities.SparseMatrixSimilarity(corpus, num_features=len(dct))
change = [1 - index[corpus[i-1]][i] for i in range(1, len(corpus))] # 0=same, 1=all-new
```

Plotting `change` over time is the timeline from Monday: spikes flag major revisions — a new policy, a redesign, a response to controversy.

Textbook Ch. 7, "Document Similarity with Bag-of-Words" & "Visualizing Document Change."

Lab: work these, then show-and-tell

Work in pairs. Do these exercises from the notebook:

1. Select a policy document. Get 5 or more versions from different dates. Plot the length of each version.
2. Use the CDX API. Compare the capture frequency and the page size of two competing organizations.
3. Calculate the similarity between all pairs of versions. Make a heatmap. Identify the points of change.
4. Examine the archive history of a government site. Report the gaps, the coverage, and the change in size.
5. Compare the earliest snapshot with the most recent snapshot. Write 200 words about the differences.
6. **5617**: assess the limits of the Wayback Machine. Include the toolbar, the capture bias, and the gaps. Compare your results with Rogers (2019)

Show-and-Tell

Bring one of these to class:

- a bug that you could not solve
- a change you found in the archived past
- a question for a research design

You are not reading old web pages.
You are measuring how institutions behaved —
one archived capture at a time.

The page you study today may be gone tomorrow:
serialize everything.

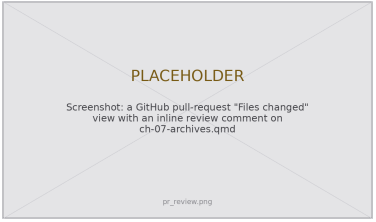
3. Friday · Textbook Revisions

Improving Chapter 7 together

Today we make the book better. Each of you opens a **pull request** against `ch-07-archives.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable



Contributions & reviews count toward the Textbook Revisions grade (15%).

A revision menu for Chapter 7

High-value targets — pick one and scope it to a single PR:

- **Run it against the live APIs.** The comparison numbers and "68 of 77 snapshots" are *illustrative*. Query the real Availability/CDX endpoints, report actual results, and label sample outputs as representative.
- **Add the chapter's first figures.** It is all code today: a Wayback calendar view, an annotated wrapped-vs-`id_` capture, or a CDX-response schema would anchor the APIs.
- **Close a gap in "Common Issues to Debug."** A worked `id_` example (link counts wrapped vs. raw), a copy-paste `nltk.download("stopwords")` recipe, or a CDX missing-field case.
- **Make an exercise self-checking.** Add expected output, a rubric, or a starter cell to the open-ended policy-tracking and heatmap exercises.
- **Promote the limitations into the main text.** Capture-frequency bias, coverage gaps, and link rewriting currently live only in the graduate exercise — tie them to "Further Reading" (Brugger et al. (2018)).

Targets drawn from the chapter's callouts, "Common Issues," exercises, & "Further Reading."

A good revision, and a good review

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Builds (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates "must fix" from "nice to have"
- Is generous — assume good faith
- Ends with a clear verdict

Next week: **Dynamic pages** — when the data appears only after JavaScript runs (Selenium).

Key takeaways

1. The web is impermanent (link rot); the **Internet Archive** is public infrastructure that preserves it.
2. **Availability API** finds one snapshot near a date; the **CDX Server** enumerates the full history — filter and collapse server-side.
3. Append `id_` for the **raw** capture; the toolbar-wrapped version pollutes every measurement.
4. Wayback + text analysis turns the archive into **longitudinal data** — measure change, don't just observe it.
5. Be a good citizen: `sleep` between requests, cache raw captures, and **serialize** — you may not be able to re-scrape.

Reading: Textbook Ch. 7. Related: Brugger et al. (2018); Rogers (2019).